

AI-Driven Fruits & Vegetables Price Forecast
& Smart Market Recommendation System

Project Type	MCA Final Year Project
Year	2026
Technology	Python Flask, XGBoost, SQLite, Bootstrap 5
Commodities	19 Agricultural Commodities
States Covered	5 Indian States, Multiple Districts
Languages	English, Telugu, Hindi, Tamil, Kannada

1. Introduction & Problem Statement

Indian farmers often face severe financial losses due to unpredictable agricultural price volatility. Prices of fruits and vegetables can swing 30–60% within a single week due to seasonal factors, weather anomalies, and supply-demand imbalances. Without access to predictive tools, farmers rely on guesswork or intermediaries who exploit this information gap.

Key Problems Addressed:

- Farmers earn significantly less than market value due to selling at unstable prices
 - No accessible district-level tool to forecast next-week prices for 19 commodities
 - Weather events (rainfall, temperature, humidity) cause sudden unexpected price swings
 - Traders exploit information asymmetry — farmers sell at wrong place & wrong time
 - No multilingual advisory system accessible to non-English-speaking rural farmers
-

2. Project Objectives

- Forecast next-week modal prices for 19 agricultural commodities using Machine Learning
 - Use XGBoost Regressor with weather + location + seasonal features as inputs
 - Build a Smart Market Recommendation engine to identify the most profitable district
 - Generate dynamic Hold vs Sell Farmer Advisory with weather-context reasoning
 - Deliver a multilingual web interface in 5 Indian languages (EN, TE, HI, TA, KN)
 - Achieve fast, responsive predictions with less than 2-second response time
 - Provide an interactive Chart.js price trend visualization for the farmer
-

3. System Architecture

The system follows a layered MVC-inspired architecture built on Flask:

Layer	Component	Description
Frontend	HTML/CSS/JS/Bootstrap 5	Glassmorphism UI, responsive design, Chart.js graphs
Auth Layer	Flask-Login + SQLite	Session management, user login/registration
Dashboard	Jinja2 Templates	Commodity / State / District dropdowns
Weather Layer	Mock Weather Engine	Generates temp, rainfall, humidity context
ML Inference	XGBoost + Joblib	Loads trained model, encodes inputs, returns price
Market Engine	Python (app.py)	Compares price across 4 districts, finds best
Advisory	generate_advisory()	Builds data-driven Hold/Sell recommendation
i18n Engine	JS + JSON files	Translates full UI into 5 Indian languages
Database	SQLite / SQLAlchemy	Stores user accounts and session data

4. Machine Learning Implementation

4.1 Dataset

- 50,000+ synthetic records generated from agricultural_data.csv
- Features: Month, Commodity (19), State (5), District (multiple), Temperature, Rainfall, Humidity
- Target Variable: Modal Price per kilogram (₹/kg)
- All categorical features encoded using Scikit-learn LabelEncoder

4.2 Model Selection & Evaluation

Algorithm	Accuracy (R ²)	Chosen?
Linear Regression	~0.71	No
Random Forest	~0.87	No
XGBoost Regressor	~0.93+	■ Yes

4.3 ML Pipeline

1. Load agricultural_data.csv with Pandas
2. Encode categorical columns: Commodity, State, District → LabelEncoder
3. Split dataset: 80% training / 20% testing
4. Train XGBoost Regressor (n_estimators=100, max_depth=6, learning_rate=0.1)
5. Evaluate R² and RMSE on test set
6. Save model: joblib.dump(model, 'xgboost_model.pkl')
7. Save encoders: joblib.dump(encoders, 'label_encoders.pkl')
8. Flask loads both files at startup via joblib.load()

5. Technology Stack

Category	Technology	Purpose
Frontend	HTML5, CSS3 (Glassmorphism)	Structure & premium UI design
CSS Framework	Bootstrap 5	Responsive grid & components
JavaScript	Vanilla JS (ES6+)	Dynamic DOM & i18n engine
Charting	Chart.js	Interactive price trend graph
Backend	Python Flask	Web framework and routing
Auth	Flask-Login	Session-based authentication
ORM	Flask-SQLAlchemy	Database abstraction layer
Templating	Jinja2	Server-side HTML rendering
ML	XGBoost + Scikit-learn	Price forecasting model
Data Processing	Pandas + NumPy	Dataset handling & features
Model Storage	Joblib	Serialize/load .pkl files
Database	SQLite	Lightweight user storage

Fonts & Icons	Google Fonts + Font Awesome	Typography and UI icons
---------------	-----------------------------	-------------------------

6. Key Features & Modules

■ Secure Authentication

User registration and login using Flask-Login with email-based identity. Passwords stored in SQLite. Sessions managed securely with Flask's secret key.

■ 19 Agricultural Commodities

Tomato, Onion, Potato, Banana, Apple, Mango, Cabbage, Carrot, Brinjal, Grapes, Orange, Papaya, Pomegranate, Watermelon, Spinach, Cauliflower, Garlic, Ginger, Green Chilli.

■ Location-Based Precision

5 Indian states (Andhra Pradesh, Telangana, Karnataka, Tamil Nadu, Maharashtra) with 5 districts each. District-level granularity for accurate local price forecasting.

■ Weather-Aware Model

Temperature (25–35°C), rainfall (10–50mm), and humidity (40–70%) integrated as features in the XGBoost model. Weather context also used in advisory reasoning.

■ Interactive Price Chart

Chart.js line graph showing 8 weeks of simulated historical prices alongside the AI-forecasted next-week price. Animated and color-coded trend display.

■ Smart Market Recommendation

After prediction, the system samples 3 random nearby districts, predicts prices for each, sorts them, and recommends the most profitable market to the farmer.

■ Dynamic Farmer Advisory

`generate_advisory()` function builds context-aware Hold vs Sell advice considering trend severity, commodity type (perishable/storable), and current weather conditions.

■ Multilingual i18n Engine

Complete 5-language support (EN/TE/HI/TA/KN) via JSON language packs and a custom JavaScript i18n engine. Language preference persists across sessions via `localStorage`.

7. Database Design

The application uses SQLite via Flask-SQLAlchemy. The database is automatically created on first run using `db.create_all()` inside the Flask app context.

Column	Type	Constraint	Description
<code>id</code>	INTEGER	PRIMARY KEY	Auto-increment user ID

name	VARCHAR(100)	NOT NULL	User's full name
email	VARCHAR(100)	UNIQUE NOT NULL	User's email address
password	VARCHAR(100)	NOT NULL	User password (plain text)

8. Project File Structure

```
mca_project/
  app.py ← Main Flask application (307 lines)
  generate_ppt.py ← PowerPoint generator
  generate_pdf.py ← PDF report generator
  requirements.txt ← Python dependencies
  agricultural_data.csv ← Training dataset (50,000+ records)
  models/
    xgboost_model.pkl ← Trained XGBoost model
    label_encoders.pkl ← Scikit-learn LabelEncoders
  templates/
    base.html ← Base layout with navbar & assets
    dashboard.html ← Prediction input form
    login.html ← Login page
    register.html ← Registration page
    results.html ← Prediction output & charts
  static/
    css/style.css ← Custom CSS (glassmorphism, animations)
    js/i18n.js ← Multilingual engine
    lang/
      en.json ← English translations
      te.json ← Telugu translations
      hi.json ← Hindi translations
      ta.json ← Tamil translations
      kn.json ← Kannada translations
  instance/
  agricultural_db.sqlite3 ← SQLite user database
```

9. Application Routes (API Endpoints)

Route	Method	Auth?	Description
/	GET	No	Redirects to dashboard (if logged in) or login
/login	GET, POST	No	User authentication — validates email & password
/register	GET, POST	No	New user registration — stores to SQLite
/logout	GET	Yes	Clears session, redirects to login
/dashboard	GET	Yes	Shows prediction form with dropdown options
/predict	POST	Yes	Runs XGBoost prediction, returns results.html

10. Conclusion & Future Work

10.1 Conclusion

- Successfully built a full-stack AI web application for agricultural price forecasting
- XGBoost Regressor trained on 50,000+ records achieves over 93% R^2 accuracy
- System covers 19 commodities across 5 major Indian agricultural states
- Smart market recommendation helps farmers identify the best-price market district
- Multilingual farmer advisory in 5 languages eliminates language barriers
- Complete Python Flask MVC architecture — modular, extensible, and maintainable

10.2 Future Enhancements

- Integration with live government pricing APIs (e-NAM, Agmarknet) for real market data
- Open-Meteo live weather API replacing the current mock weather engine
- SMS / WhatsApp alert system for sudden price changes using Twilio integration
- Mobile-first Progressive Web App (PWA) with offline support for rural farmers
- Expand to 100+ commodities and cover all 28+ Indian states and UTs
- Crop planting season advisor — recommend what to plant based on predicted prices
- Machine learning model retraining pipeline with periodic fresh data ingestion
- Admin dashboard for monitoring user activity and forecast accuracy over time